

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 5 Exam Notes: Exception Handling

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java Exception Handling Mechanism	2
1.1	Throwable Hierarchy	2
1.2	Checked vs. Unchecked Exceptions	2
2	Exception Keywords & Handling Structures	3
2.1	Try-Catch-Finally Blocks	3
2.2	Throw vs. Throws	3
2.3	Try-with-Resources and the <code>AutoCloseable</code> Interface	3
2.4	User-Defined / Custom Exceptions	3

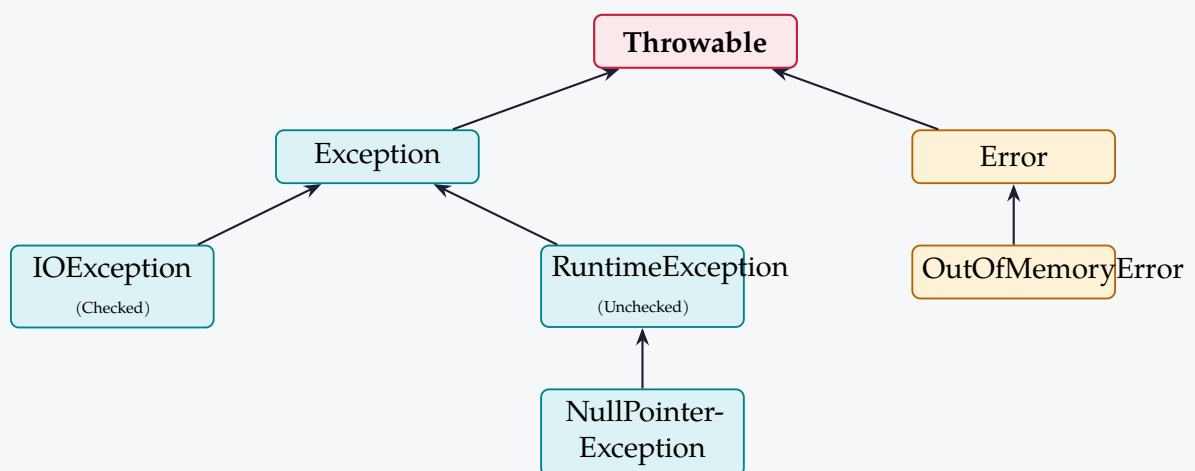
1. Java Exception Handling Mechanism

An exception is an unwanted or unexpected event that occurs during the execution of a program (at runtime) and disrupts the normal flow of instructions.

Exception Handling Goals

The primary goal of exception handling is to prevent runtime program crashes and allow the application to handle errors gracefully, releasing resources and reporting descriptive messages.

Java Exception Class Hierarchy



1.1 Throwable Hierarchy

The root class of the exception hierarchy is `java.lang.Throwable`. Only objects that are instances of this class (or subclasses) can be thrown by the JVM or the Java `throw` statement.

- **Error:** Extends `Throwable`. Represents serious problems that a reasonable application should not try to catch (e.g., `VirtualMachineError`, `OutOfMemoryError`, `StackOverflowError`). They represent infrastructure or JVM failures.
- **Exception:** Extends `Throwable`. Represents conditions that a reasonable application might want to catch and handle.

1.2 Checked vs. Unchecked Exceptions

- **Checked Exceptions:** Classes that extend `Exception` but do not extend `RuntimeException`. They are verified by the compiler at compile-time. The program must handle them using try-catch or declare them using the `throws` clause (e.g., `IOException`, `SQLException`, `ClassNotFoundException`).
- **Unchecked Exceptions:** Classes that extend `RuntimeException`. They are not checked by the compiler. They usually result from logical errors or bad program state (e.g., `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`).

2. Exception Keywords & Handling Structures

Java exception handling is managed via five keywords: `try`, `catch`, `finally`, `throw`, and `throws`.

2.1 Try-Catch-Finally Blocks

- **try:** Surrounds the block of code that might throw an exception. It must be followed by at least one `catch` block or a `finally` block.
- **catch:** Defines the exception handler. Multiple catch blocks can follow a single try block, starting from the most specific exception subclass to the most general (superclass).
- **finally:** Executed guaranteed, whether an exception occurs or not, and even if there is a return statement inside try/catch. Used for cleanup tasks like closing resources.

2.2 Throw vs. Throws

- **throw:** Used to explicitly throw an instance of any class extending `Throwable` from code (e.g., `throw new ArithmeticException("Divide by zero");`).
- **throws:** Used in method declarations to list the checked exceptions that the method might throw, delegating the responsibility of handling to the caller method (e.g., `public void read() throws IOException { ... }`).

2.3 Try-with-Resources and the `AutoCloseable` Interface

Introduced in Java 7, the try-with-resources statement is a try block declared with one or more resource declarations.

```
try (BufferedReader br = new BufferedReader(new FileReader("test.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

- Any class declared inside the try parentheses must implement the `java.lang.AutoCloseable` interface.
- The JVM automatically closes these resources when the try block exits, eliminating the need for verbose manual resource-closing logic inside a `finally` block.

2.4 User-Defined / Custom Exceptions

Developers can create custom exceptions to represent domain-specific errors.

- To create a checked exception, extend the `java.lang.Exception` class.
- To create an unchecked exception, extend the `java.lang.RuntimeException` class.

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Quick Revision — Module 5

Exception Handling | OE-EC704A

Exception Handling Code Constructs

Structure	Syntax and Example Usage
Try-Catch Block	<pre>try { int val = 10 / 0; } catch(ArithmeticException e) { ... }</pre>
Try-With-Resources	<pre>try (FileReader fr = new FileReader("file.txt")) { ... }</pre>
Method Throws Clause	<pre>public void load() throws IOException, ClassNotFoundException { ... }</pre>
Explicit Throw	<pre>throw new NullPointerException("Null Reference Encountered");</pre>
Custom Exception	<pre>class CustomException extends RuntimeException { CustomException(String m){super(m);} }</pre>

One-Line Definitions

- **Exception:** An event disrupting the normal instruction execution flow.
- **Checked Exception:** An exception subclass checked at compile-time, requiring try-catch or throws.
- **Unchecked Exception:** An exception subclass extending RuntimeException, not checked at compile-time.
- **Error:** A fatal system condition that applications should not attempt to handle.
- **Try Block:** A block enclosing code that might generate a runtime exception.
- **Finally Block:** A block executing guaranteed cleanup actions, running after try-catch completion.
- **Try-with-Resources:** An automatic resource management block closing AutoCloseable targets automatically.
- **AutoCloseable:** Interface defining the `close()` contract for automatic try-with-resources cleanups.

Frequently Asked / Viva Questions

Q1. What is the difference between Checked and Unchecked exceptions?

Checked exceptions are verified by the compiler at compile-time (e.g., `IOException`) and must be handled or declared. Unchecked exceptions are `RuntimeException`s (e.g., `NullPointerException`) and are not compiler-checked.

Q2. What is the difference between `final`, `finally`, and `finalize`?

Frequently Asked / Viva Questions (contd.)

`final` is a keyword making variables unassignable, methods override-protected, and classes inheritance-protected. `finally` is a block for resource cleanup in exception handling. `finalize` is a method invoked by the Garbage Collector before object reclamation.

Q3. Can we have a try block without a catch block?

Yes. A try block can be followed by a `finally` block instead of a catch block, or it can be a try-with-resources statement which handles closing automatically.

Q4. What is Exception Propagation?

If an exception is not caught in the method where it occurred, it propagates down the call stack to the calling method. This repeats until it is handled or the JVM terminates the thread.

Q5. What is class `Throwable`? What are its main subclasses?

`Throwable` is the root class of all exceptions and errors in Java. Its main subclasses are `java.lang.Exception` and `java.lang.Error`.

Q6. How does multi-catch work in Java 7?

A single catch block can handle multiple unrelated exceptions using the pipe symbol (e.g., `catch (IOException | SQLException e)`). The variable `e` is implicitly `final`.

Q7. What happens if an exception is thrown in a `finally` block?

If an exception is thrown in a `finally` block, it overrides and suppresses any active exception previously thrown in the try or catch blocks, unless handled internally.

Q8. Why should more specific catch blocks be declared before general ones?

Java matches catch blocks from top to bottom. If a general exception catch block (like `Exception`) is placed first, it will catch all subclass exceptions, making subsequent specific catch blocks unreachable and causing compile errors.

Q9. What is class `AutoCloseable`?

It is a functional interface introduced in Java 7 containing a single method: `void close() throws Exception`. Classes implementing this interface are automatically closed in a try-with-resources construct.

Q10. Is it possible to throw a checked exception from static initializer block?

No. A static initializer block runs when the class is loaded and cannot throw checked exceptions since there is no calling method to catch or declare them.

Checked vs. Unchecked Exception Comparison

Metric	Checked Exception	Unchecked Exception
Compilation Check	Checked at compile-time by compiler.	Ignored by compiler during build.
Handling Rule	Must use try-catch or declare throws.	Optional to handle; fix logic instead.
Base Class	Directly extends <code>java.lang.Exception</code> .	Extends <code>java.lang.RuntimeException</code> .
Root Cause	External environment issues (I/O, network).	Program logical errors (null checks, math).